

Índice

1	C++	5
1.1	Compilador	5
1.2	STL (Standard Template Library)	6
1.2.1	Vector	6
1.2.2	Pair	7
1.2.3	Set	7
1.2.4	Multiset	7
1.2.5	Map	8
1.2.6	Queue	8
1.2.7	Priority Queue	9
1.2.8	Stack	9
1.2.9	Bitset	10
1.2.10	Funções úteis	10
1.2.11	Funções úteis para vetores	11
1.3	Pragmas	12
1.4	Constantes em C++	13

ÍNDICE	2
2 Teórico	14
2.1 Definições	14
2.2 Números primos	14
2.2.1 Primos com truncamento à esquerda	14
2.2.2 Primos gêmeos (Twin Primes)	15
2.2.3 Números primos de Mersenne	15
2.3 Operadores lineares	16
2.3.1 Rotação no sentido anti-horário por θ°	16
2.3.2 Reflexão em relação à reta $y = mx$	16
2.3.3 Inversa de uma matriz 2x2 A	16
2.3.4 Cisalhamento horizontal por K	16
2.3.5 Cisalhamento vertical por K	16
2.3.6 Mudança de base	16
2.3.7 Propriedades das operações de matriz	17
2.4 Sequências numéricas	17
2.4.1 Sequência de Fibonacci	17
2.4.2 Sequência de Catalan	18
2.5 Análise combinatória	19
2.5.1 Fatorial	19
2.5.2 Combinação	19
2.5.3 Arranjo	19
2.5.4 Estrelas e barras	19
2.5.5 Princípio da inclusão-exclusão	20
2.5.6 Princípio da casa dos pombos	20
2.6 Teoria dos números	20

<i>ÍNDICE</i>	3
2.6.1 Pequeno teorema de Fermat	20
2.6.2 Teorema de Euler	20
2.6.3 Aritmética modular	20
3 Extra	22
4 Algoritmos	24
4.1 Histograma	24
5 ds	26
5.1 Heavy Light Decomposition	26
5.2 Longest Increasing Subsequence	27
5.3 MinMaxQueue	28
5.4 SegTree	29
5.5 Sparse Table Disjunta	35
6 geometry	37
6.1 Convex Hull	37
7 graph	39
7.1 Floyd warshall	39
7.2 Max Flow Min Cost	40
7.3 Push Relabel	42
8 math	44
8.1 Brent	44
8.2 FFT	45
8.3 Fermat	49

ÍNDICE

4

8.4 Miller Rabin

50

8.5 Pollard Rho

50

8.6 Teorema chines

51

8.7 Utils

52

9 strings

53

9.1 Suffix Array

53

Capítulo 1

C++

1.1 Compilador

Para compilar um arquivo `.cpp` com o compilador `g++`, usar o comando:

```
1 g++ -std=c++20 -O2 arquivo.cpp
```

Obs: a flag `-std=c++20` é para usar a versão 20 do C++, os códigos desse Almanaque são testados com essa versão.

Algumas flags úteis para o `g++` são:

- `-O2`: Otimizações de compilação
- `-Wall`: Mostra todos os warnings
- `-Wextra`: Mostra mais warnings
- `-Wconversion`: Mostra warnings para conversões implícitas
- `-fsanitize=address`: Habilita o AddressSanitizer
- `-fsanitize=undefined`: Habilita o UndefinedBehaviorSanitizer

Todas essas flags já estão presente no script ‘run’ da seção Extra.

1.2 STL (Standard Template Library)

Os templates da STL são estruturas de dados e algoritmos já implementadas em C++ que facilitam as implementações, além de serem muito eficientes. Em geral, todas estão incluídas no cabeçalho `<bits/stdc++.h>`. As estruturas são templates genéricos, podem ser usadas com qualquer tipo, todos os exemplos a seguir são com `int` apenas por motivos de simplicidade.

1.2.1 Vector

Um vetor dinâmico (que pode crescer e diminuir de tamanho).

- `vector<int> v(n, x)`: Cria um vetor de inteiros com `n` elementos, todos inicializados com `x` - $\mathcal{O}(n)$
- `v.push_back(x)`: Adiciona o elemento `x` no final do vetor - $\mathcal{O}(1)$
- `v.pop_back()`: Remove o último elemento do vetor - $\mathcal{O}(1)$
- `v.size()`: Retorna o tamanho do vetor - $\mathcal{O}(1)$
- `v.empty()`: Retorna `true` se o vetor estiver vazio - $\mathcal{O}(1)$
- `v.clear()`: Remove todos os elementos do vetor - $\mathcal{O}(n)$
- `v.front()`: Retorna o primeiro elemento do vetor - $\mathcal{O}(1)$
- `v.back()`: Retorna o último elemento do vetor - $\mathcal{O}(1)$
- `v.begin()`: Retorna um iterador para o primeiro elemento do vetor - $\mathcal{O}(1)$
- `v.end()`: Retorna um iterador para o elemento seguinte ao último do vetor - $\mathcal{O}(1)$
- `v.insert(it, x)`: Insere o elemento `x` na posição apontada pelo iterador `it` - $\mathcal{O}(n)$
- `v.erase(it)`: Remove o elemento apontado pelo iterador `it` - $\mathcal{O}(n)$
- `v.erase(it1, it2)`: Remove os elementos no intervalo `[it1, it2)` - $\mathcal{O}(n)$
- `v.resize(n)`: Redimensiona o vetor para `n` elementos - $\mathcal{O}(n)$
- `v.resize(n, x)`: Redimensiona o vetor para `n` elementos, todos inicializados com `x` - $\mathcal{O}(n)$

1.2.2 Pair

Um par de elementos (de tipos possivelmente diferentes).

- `pair<int, int> p`: Cria um par de inteiros - $\mathcal{O}(1)$
- `p.first`: Retorna o primeiro elemento do par - $\mathcal{O}(1)$
- `p.second`: Retorna o segundo elemento do par - $\mathcal{O}(1)$

1.2.3 Set

Um conjunto de elementos únicos. Por baixo, é uma árvore de busca binária balanceada.

- `set<int> s`: Cria um conjunto de inteiros - $\mathcal{O}(1)$
- `s.insert(x)`: Insere o elemento `x` no conjunto - $\mathcal{O}(\log n)$
- `s.erase(x)`: Remove o elemento `x` do conjunto - $\mathcal{O}(\log n)$
- `s.find(x)`: Retorna um iterador para o elemento `x` no conjunto, ou `s.end()` se não existir - $\mathcal{O}(\log n)$
- `s.size()`: Retorna o tamanho do conjunto - $\mathcal{O}(1)$
- `s.empty()`: Retorna `true` se o conjunto estiver vazio - $\mathcal{O}(1)$
- `s.clear()`: Remove todos os elementos do conjunto - $\mathcal{O}(n)$
- `s.begin()`: Retorna um iterador para o primeiro elemento do conjunto - $\mathcal{O}(1)$
- `s.end()`: Retorna um iterador para o elemento seguinte ao último do conjunto - $\mathcal{O}(1)$

1.2.4 Multiset

Basicamente um `set`, mas permite elementos repetidos. Possui todos os métodos de um `set`.

Declaração: `multiset<int> ms`.

Um detalhe é que, ao usar o método `erase`, ele remove todas as ocorrências do elemento. Para remover apenas uma ocorrência, usar `ms.erase(ms.find(x))`.

1.2.5 Map

Um conjunto de pares chave-valor, onde as chaves são únicas. Por baixo, é uma árvore de busca binária balanceada.

- `map<int, int> m`: Cria um mapa de inteiros para inteiros - $\mathcal{O}(1)$
- `m[key]`: Retorna o valor associado à chave `key` - $\mathcal{O}(\log n)$
- `m[key] = value`: Associa o valor `value` à chave `key` - $\mathcal{O}(\log n)$
- `m.erase(key)`: Remove a chave `key` do mapa - $\mathcal{O}(\log n)$
- `m.find(key)`: Retorna um iterador para o par chave-valor com chave `key`, ou `m.end()` se não existir - $\mathcal{O}(\log n)$
- `m.size()`: Retorna o tamanho do mapa - $\mathcal{O}(1)$
- `m.empty()`: Retorna `true` se o mapa estiver vazio - $\mathcal{O}(1)$
- `m.clear()`: Remove todos os pares chave-valor do mapa - $\mathcal{O}(n)$
- `m.begin()`: Retorna um iterador para o primeiro par chave-valor do mapa - $\mathcal{O}(1)$
- `m.end()`: Retorna um iterador para o par chave-valor seguinte ao último do mapa - $\mathcal{O}(1)$

1.2.6 Queue

Uma fila (primeiro a entrar, primeiro a sair).

- `queue<int> q`: Cria uma fila de inteiros - $\mathcal{O}(1)$
- `q.push(x)`: Adiciona o elemento `x` no final da fila - $\mathcal{O}(1)$
- `q.pop()`: Remove o primeiro elemento da fila - $\mathcal{O}(1)$
- `q.front()`: Retorna o primeiro elemento da fila - $\mathcal{O}(1)$
- `q.size()`: Retorna o tamanho da fila - $\mathcal{O}(1)$
- `q.empty()`: Retorna `true` se a fila estiver vazia - $\mathcal{O}(1)$

1.2.7 Priority Queue

Uma fila de prioridade (o maior elemento é o primeiro a sair).

- `priority_queue<int> pq`: Cria uma fila de prioridade de inteiros - $\mathcal{O}(1)$
- `pq.push(x)`: Adiciona o elemento `x` na fila de prioridade - $\mathcal{O}(\log n)$
- `pq.pop()`: Remove o maior elemento da fila de prioridade - $\mathcal{O}(\log n)$
- `pq.top()`: Retorna o maior elemento da fila de prioridade - $\mathcal{O}(1)$
- `pq.size()`: Retorna o tamanho da fila de prioridade - $\mathcal{O}(1)$
- `pq.empty()`: Retorna `true` se a fila de prioridade estiver vazia - $\mathcal{O}(1)$

Para fazer uma fila de prioridade que o menor elemento é o primeiro a sair, usar `priority_queue< int, vector<int>, greater<> > pq`.

1.2.8 Stack

Uma pilha (último a entrar, primeiro a sair).

- `stack<int> s`: Cria uma pilha de inteiros - $\mathcal{O}(1)$
- `s.push(x)`: Adiciona o elemento `x` no topo da pilha - $\mathcal{O}(1)$
- `s.pop()`: Remove o elemento do topo da pilha - $\mathcal{O}(1)$
- `s.top()`: Retorna o elemento do topo da pilha - $\mathcal{O}(1)$
- `s.size()`: Retorna o tamanho da pilha - $\mathcal{O}(1)$
- `s.empty()`: Retorna `true` se a pilha estiver vazia - $\mathcal{O}(1)$

1.2.9 Bitset

Um conjunto de bits, serve para representar máscaras quando um inteiro não é suficiente. Possui operações bitwise otimizadas pelo processador.

- `bitset<N> a`: Cria um bitset de tamanho `N` (`N` deve ser constante) - $\mathcal{O}(1)$
- `a[i]`: Retorna o valor do bit na posição `i` - $\mathcal{O}(1)$
- `a.count()`: Retorna o número de bits 1 no bitset - $\mathcal{O}(N/w)$
- `a.size()`: Retorna o tamanho do bitset - $\mathcal{O}(1)$
- `a.set(i)`: Seta o bit na posição `i` para 1 - $\mathcal{O}(1)$
- `a.set()`: Seta todos os bits para 1 - $\mathcal{O}(N/w)$
- `a.reset(i)`: Seta o bit na posição `i` para 0 - $\mathcal{O}(1)$
- `a.reset()`: Seta todos os bits para 0 - $\mathcal{O}(N/w)$
- `a.flip(i)`: Inverte o bit na posição `i` - $\mathcal{O}(1)$
- `a.flip()`: Inverte todos os bits - $\mathcal{O}(N/w)$
- `a.to_ullong()`: Retorna o valor do bitset como um inteiro - $\mathcal{O}(N/w)$

O bitset também suporta operações como `&`, `|`, `^`, `~`, `<<`, `>>`, entre outros.

OBS: w é o tamanho da palavra do processador, em geral 32 ou 64 bits.

1.2.10 Funções úteis

- `min(a, b)`: Retorna o menor entre `a` e `b` - $\mathcal{O}(1)$
- `max(a, b)`: Retorna o maior entre `a` e `b` - $\mathcal{O}(1)$
- `abs(a)`: Retorna o valor absoluto de `a` - $\mathcal{O}(1)$
- `swap(a, b)`: Troca os valores de `a` e `b` - $\mathcal{O}(1)$
- `sqrt(a)`: Retorna a raiz quadrada de `a` - $\mathcal{O}(\log a)$

- `ceil(a)`: Retorna o menor inteiro maior ou igual a `a` - $\mathcal{O}(1)$
- `floor(a)`: Retorna o maior inteiro menor ou igual a `a` - $\mathcal{O}(1)$
- `round(a)`: Retorna o inteiro mais próximo de `a` - $\mathcal{O}(1)$

1.2.11 Funções úteis para vetores

Para usar em `std::vector`, sempre passar `v.begin()` e `v.end()` como argumentos pra essas funções.

Se for um vetor estilo C, usar `v` e `v + n`. Exemplo:

```
1  int v[10];
2  sort(v, v + 10);
```

Lembrete: `v.end()` é um iterador para o elemento seguinte ao último do vetor, então não é um iterador válido.

As funções de vetor em geral são da forma `[L, R)`, ou seja, `L` é inclusivo e `R` é exclusivo.

- `fill(v.begin(), v.end(), x)`: Preenche o vetor `v` com o valor `x` - $\mathcal{O}(n)$
- `sort(v.begin(), v.end())`: Ordena o vetor `v` - $\mathcal{O}(n \log n)$
- `reverse(v.begin(), v.end())`: Inverte o vetor `v` - $\mathcal{O}(n)$
- `accumulate(v.begin(), v.end(), 0)`: Soma todos os elementos do vetor `v` - $\mathcal{O}(n)$
- `max_element(v.begin(), v.end())`: Retorna um iterador para o maior elemento do vetor `v` - $\mathcal{O}(n)$
- `min_element(v.begin(), v.end())`: Retorna um iterador para o menor elemento do vetor `v` - $\mathcal{O}(n)$
- `count(v.begin(), v.end(), x)`: Retorna o número de ocorrências do elemento `x` no vetor `v` - $\mathcal{O}(n)$
- `find(v.begin(), v.end(), x)`: Retorna um iterador para a primeira ocorrência do elemento `x` no vetor `v`, ou `v.end()` se não existir - $\mathcal{O}(n)$
- `lower_bound(v.begin(), v.end(), x)`: Retorna um iterador para o primeiro elemento maior ou igual a `x` no vetor `v` (o vetor deve estar ordenado) - $\mathcal{O}(\log n)$
- `upper_bound(v.begin(), v.end(), x)`: Retorna um iterador para o primeiro elemento estritamente maior que `x` no vetor `v` (o vetor deve estar ordenado) - $\mathcal{O}(\log n)$
- `next_permutation(a.begin(), a.end())`: Rearranja os elementos do vetor `a` para a próxima permutação lexicograficamente maior - $\mathcal{O}(n)$

1.3 Pragmas

Os pragmas são diretivas para o compilador, que podem ser usadas para otimizar o código.

Temos os pragmas de otimização, como por exemplo:

- `#pragma GCC optimize("O2")`: Otimizações de nível 2 (padrão de competições)
- `#pragma GCC optimize("O3")`: Otimizações de nível 3 (seguro para usar)
- `#pragma GCC optimize("Ofast")`: Otimizações agressivas (perigoso!)
- `#pragma GCC optimize("unroll-loops")`: Otimiza os loops mas pode levar a cache misses

E também os pragmas de target, que são usados para otimizar o código para um certo processador:

- `#pragma GCC target("avx2")`: Otimiza instruções para processadores com suporte a AVX2
- `#pragma GCC target("sse4")`: Parecido com o de cima, mas mais antigo
- `#pragma GCC target("popcnt")`: Otimiza o popcount em processadores que suportam
- `#pragma GCC target("lzcnt")`: Otimiza o leading zero count em processadores que suportam
- `#pragma GCC target("bmi")`: Otimiza instruções de bit manipulation em processadores que suportam
- `#pragma GCC target("bmi2")`: Mesmo que o de cima, mas mais recente

Em geral, esses pragmas são usados para otimizar o código em competições, mas é importante usá-los com certa sabedoria, em alguns casos eles podem piorar o desempenho do código.

Uma opção relativamente segura de se usar é a seguinte:

```
1  #pragma GCC optimize("O3,unroll-loops")
2  #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

1.4 Constantes em C++

Constante	Nome em C++	Valor
π	M_PI	3.141592...
$\pi/2$	M_PI_2	1.570796...
$\pi/4$	M_PI_4	0.785398...
$1/\pi$	M_1_PI	0.318309...
$2/\pi$	M_2_PI	0.636619...
$2/\sqrt{\pi}$	M_2_SQRTPI	1.128379...
$\sqrt{2}$	M_SQRT2	1.414213...
$1/\sqrt{2}$	M_SQRT1_2	0.707106...
e	M_E	2.718281...
$\log_2 e$	M_LOG2E	1.442695...
$\log_{10} e$	M_LOG10E	0.434294...
$\ln 2$	M_LN2	0.693147...
$\ln 10$	M_LN10	2.302585...

Capítulo 2

Teórico

2.1 Definições

Algumas definições e termos importantes:

2.2 Números primos

Números primos são muito úteis para funções de hashing (dentre outras coisas). Aqui vão vários números primos interessantes:

2.2.1 Primos com truncamento à esquerda

Números primos tais que qualquer sufixo deles é um número primo:

33,333,31
357,686,312,646,216,567,629,137

2.2.2 Primos gêmeos (Twin Primes)

Pares de primos da forma $(p, p + 2)$ (aqui tem só alguns pares aleatórios, existem muitos outros).

Primo	Primo + 2	Ordem
5	7	10^0
17	19	10^1
461	463	10^2
3461	3463	10^3
34499	34501	10^4
487829	487831	10^5
5111999	5112001	10^6
30684887	30684889	10^7
361290539	361290541	10^8
1000000007	1000000009	10^9
1005599459	1005599461	10^9

2.2.3 Números primos de Mersenne

São os números primos da forma $2^m - 1$, onde m é um número inteiro positivo.

Expoente (m)	Representação Decimal
2	3
3	7
5	31
7	127
13	8,191
17	131,071
19	524,287
31	2,147,483,647
61	$2,3 * 10^{18}$
89	$6,1 * 10^{26}$
107	$1,6 * 10^{32}$
127	$1,7 * 10^{38}$

2.3 Operadores lineares

2.3.1 Rotação no sentido anti-horário por θ°

$$\begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

2.3.2 Reflexão em relação à reta $y = mx$

$$\frac{1}{m^2 + 1} \begin{bmatrix} 1 - m^2 & 2m \\ 2m & m^2 - 1 \end{bmatrix}$$

2.3.3 Inversa de uma matriz 2x2 A

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{\det(A)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

2.3.4 Cisalhamento horizontal por K

$$\begin{bmatrix} 1 & K \\ 0 & 1 \end{bmatrix}$$

2.3.5 Cisalhamento vertical por K

$$\begin{bmatrix} 1 & 0 \\ K & 1 \end{bmatrix}$$

2.3.6 Mudança de base

$\vec{a}_\beta$ são as coordenadas do vetor $\vec{a}$ na base β .

$\vec{a}$ são as coordenadas do vetor $\vec{a}$ na base canônica.

$\vec{b1}$ e $\vec{b2}$ são os vetores de base para β .

C é uma matriz que muda da base β para a base canônica.

$$C\vec{a}_\beta = \vec{a}$$

$$C^{-1}\vec{a} = \vec{a}_\beta$$

$$C = \begin{bmatrix} b1_x & b2_x \\ b1_y & b2_y \end{bmatrix}$$

2.3.7 Propriedades das operações de matriz

$$(AB)^{-1} = A^{-1}B^{-1}$$

$$(AB)^T = B^T A^T$$

$$(A^{-1})^T = (A^T)^{-1}$$

$$(A + B)^T = A^T + B^T$$

$$\det(A) = \det(A^T)$$

$$\det(AB) = \det(A)\det(B)$$

Seja A uma matriz NxN:

$$\det(kA) = K^N \det(A)$$

2.4 Sequências numéricas

2.4.1 Sequência de Fibonacci

Primeiros termos: **1, 1, 2, 3, 5, 8, 13, 21, 34, ...**

Definição:

$$F_0 = F_1 = 1$$

$$F_n = F_{n-1} + F_{n-2}$$

Matriz de recorrência:

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} F_{n-2} \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} F_{n-1} \\ F_n \end{bmatrix}$$

2.4.2 Sequência de Catalan

Primeiros termos: **1, 1, 2, 5, 14, 42, 132, 429, 1430, ...**

Definição:

$$C_0 = C_1 = 1$$

$$C_n = \sum_{i=0}^{n-1} C_i \cdot C_{n-1-i}$$

Definição analítica:

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

Propriedades úteis:

- C_n é o número de árvores binárias com $n+1$ folhas.
- C_n é o número de sequências de parênteses bem formadas com n pares de parênteses.

2.5 Análise combinatória

2.5.1 Fatorial

O fatorial de um número n é o produto de todos os inteiros positivos menores ou iguais a n .

O fatorial conta o número de permutações de n elementos.

$$n! = n \cdot (n - 1)!$$

Em particular, $0! = 1$.

2.5.2 Combinação

Conta o número de maneiras de escolher k elementos de um conjunto de n elementos.

$$\binom{n}{k} = \frac{n!}{k! \cdot (n - k)!}$$

2.5.3 Arranjo

Conta o número de maneiras de escolher k elementos de um conjunto de n elementos, onde a ordem importa.

$$P(n, k) = \frac{n!}{(n - k)!}$$

2.5.4 Estrelas e barras

Conta o número de maneiras de distribuir n elementos idênticos em k recipientes distintos.

$$\binom{n + k - 1}{k - 1}$$

2.5.5 Princípio da inclusão-exclusão

O princípio da inclusão-exclusão é uma técnica para contar o número de elementos em uma união de conjuntos.

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{k=1}^n (-1)^{k+1} \left(\sum_{1 \leq i_1 < i_2 < \dots < i_k \leq n} |A_{i_1} \cap A_{i_2} \cap \dots \cap A_{i_k}| \right)$$

2.5.6 Princípio da casa dos pombos

Se n pombos são colocados em m casas, então pelo menos uma casa terá $\lceil \frac{n}{m} \rceil$ pombos ou mais.

2.6 Teoria dos números

2.6.1 Pequeno teorema de Fermat

Se p é um número primo e a é um inteiro não divisível por p , então $a^{p-1} \equiv 1 \pmod{p}$.

2.6.2 Teorema de Euler

Se m e a são inteiros positivos coprimos, então $a^{\phi(m)} \equiv 1 \pmod{m}$, onde $\phi(m)$ é a função totiente de Euler.

2.6.3 Aritmética modular

Quando estamos trabalhando com aritmética módulo um número p , todos os valores existentes estão entre $[0, p-1]$.

Algumas propriedades e equivalências úteis para usar aritmética modular em código são:

- $(a + b) \pmod{p} \equiv ((a \pmod{p}) + (b \pmod{p})) \pmod{p}$
- $(a - b) \pmod{p} \equiv ((a \pmod{p}) - (b \pmod{p})) \pmod{p}$

– Note que o resultado pode ser negativo, nesse é necessário adicionar p ao resultado. De forma geral, geralmente fazemos $(a - b + p) \pmod{p}$ (assumindo que a e b já estão no intervalo $[0, p-1]$).

- $(a \cdot b) \bmod p \equiv ((a \bmod p) \cdot (b \bmod p)) \bmod p$
- $a^b \bmod p \equiv ((a \bmod p)^b) \bmod p$
- $a^b \bmod p \equiv ((a \bmod p)^{(b \bmod \phi(p))}) \bmod p$ se a e p são coprimos

Capítulo 3

Extra

Extra

Codigo: template.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long int ll;
4 typedef pair<int,int> ii;
5 typedef vector<int> vi;
6 typedef vector<ll> vll;
7 #define INF (1<<30)
8 #define x first
9 #define y second
10 #define pb push_back
11 #define mp make_pair
12 #define lsb(i) (i&(-i))
13 #define all(z) begin(z), end(z)
14 #define EPS ((double)(1e-9))
15 #define MOD 1000000007
16 const int maxn = 1123456;
17
18 void solve(){
19
20 }
21
22
23 int main(){
```

```
24 ios_base::sync_with_stdio(0);
25 cin.tie(0)->tie(0);
26 int t = 1;
27 //cin >> t;
28
29 for(int caso=1; caso<=t; caso++){
30     solve();
31 }
32 return 0;
33 }
```

Capítulo 4

Algoritmos

4.1 Histograma

Histograma

Codigo: histogram.cpp

```
1 ll ans = 0;
2 vll esq(m+2);
3 vll dir(m+2);
4
5 S[0] = -1; // 1 -> m
6 S[m+1] = -1;
7 vector<ll> sta;
8 sta.pb(0);
9 for(int j=1;j<=m;j++){
10     while(S[sta.back()] >= S[j])
11         sta.pop_back();
12     esq[j] = sta.back();
13     sta.pb(j);
14 }
15 sta.clear();
16 sta.pb(m+1);
17 for(int j=m;j>=1;j--){
18     while(S[sta.back()] >= S[j])
19         sta.pop_back();
20     dir[j] = sta.back();
```

```
21     sta.pb(j);
22 }
23 for(int j=1;j<=m;j++){ // Maybe only one side is enough
24     ans = max(ans, (dir[j] - esq[j]) * S[j]);
25 }
```

Capítulo 5

ds

5.1 Heavy Light Decomposition

Heavy Light Decomposition

Codigo: hld.cpp

```
1 struct HLD{
2     int cnt = 0;
3     vector<int> prox[maxn];
4     int pai[maxn];
5     int siz[maxn];
6     int pos[maxn];
7     int h[maxn];
8
9     void addEdge(int u, int v){
10         prox[u].push_back(v);
11         prox[v].push_back(u);
12     }
13
14     void build(int root){
15         dfs(root);
16         cnt = 0;
17         buildHld(root);
18         seg.build(0, cnt-1);
19     }
20 }
```

```

21 void dfs(int v, int p=-1){
22     pai[v] = p;
23     siz[v] = 1;
24     for(int &w: prox[v]){
25         if(w==p) continue;
26         dfs(w,v);
27         siz[v] += siz[w];
28         if(siz[w] > siz[prox[v][0]] or prox[v][0]==p)
29             swap(w, prox[v][0]);
30     }
31 }
32 void buildHld(int v, int p=-1){
33     pos[v] = cnt++;
34     for(int &w: prox[v]){
35         if(w == p) continue;
36         h[w] = (w==prox[v][0]?h[v]:w);
37         buildHld(w, v);
38     }
39 }
40
41 ll query(int a, int b){
42     //cout << "on query "<<a <<"<<b <<endl;
43     if(pos[a] > pos[b]) swap(a,b);
44     if(h[a] == h[b])
45         return seg.query(pos[a], pos[b]);
46     return seg.query(pos[h[b]], pos[b]) + query(a, pai[h[b]]);
47 }
48 void update(int a, int b, ll x){
49     //cout << "on update "<<a <<"<<b <<"<<x<<endl;
50     if(pos[a] > pos[b]) swap(a,b);
51     if(h[a] == h[b]){
52         seg.update(pos[a], pos[b], x);
53         return;
54     }
55     seg.update(pos[h[b]], pos[b], x);
56     update(a, pai[h[b]], x);
57 }
58 }hld;

```

5.2 Longest Increasing Subsequence

Longest Increasing Subsequence

Codigo: lis.cpp

```

1 // Where to put this code?
2 vi lis[maxn];
3 void add(int x, int id){
4     int l = 0;
5     int r = lis[id].size()-1;
6     int last = lis[id].size();
7     while(l<=r){
8         int m = (l+r)/2;
9         if(lis[id][m] >= x){
10             last = min(last, m);
11             r = m - 1;
12         }
13         else l = m+1;
14     }
15     if(last>=lis[id].size()) lis[id].push_back(x);
16     else lis[id][last] = x;
17 }

```

5.3 MinMaxQueue

Min/Max Queue

Codigo: MinMaxQueue.cpp

```

1 template<typename T>
2 T fun(const T &a, const T &b){
3     return max(a,b);
4 }
5
6 template<typename T>
7 struct MMStack{
8     vector<pair<T,T>> sta;
9     size_t size(){
10         return sta.size();
11     }
12     void push(T x){
13         T mini = x;
14         if(sta.size()) mini = fun(mini, sta.back().y);
15         sta.push_back(make_pair(x, mini));
16     }
17     pair<T,T> get(){
18         return sta.back();

```

```

19     }
20     void pop(){
21         sta.pop_back();
22     }
23 };
24
25 template<typename T>
26 struct MMQueue{
27     MMStack<T> a,b;
28     T sentinel;
29     void push(T x){
30         b.push(x);
31     }
32     T get(){
33         if(a.size()==0 && b.size()==0) return sentinel;
34
35         if(a.size()==0 || b.size()==0)
36             return a.size()==0 ? b.get().y : a.get().y;
37
38         return fun(a.get().y, b.get().y);
39     }
40     T pop(){
41         if(a.size()==0){
42             while(b.size()){
43                 a.push(b.get().x);
44                 b.pop();
45             }
46         }
47         if(a.size()){
48             T response = a.get().y;
49             a.pop();
50             return response;
51         }
52         return sentinel;
53     }
54     size_t size(){
55         return a.size() + b.size();
56     }
57 };

```

5.4 SegTree

Segment Tree

Codigo: segtree2D.cpp

```

1  const int maxn = 4123;
2
3  template<typename T, int N, int M, T v[N][M]>
4  struct Seg2D{
5      int ix[N], iy[M];
6      int fx[N], fy[M];
7      T value[N][M];
8      T iden;
9      inline T fun(T a, T b){
10         return a+b;
11     }
12     void build_x(int lx, int rx, int ly, int ry, int idx=1){
13         ix[idx] = lx;
14         fx[idx] = rx;
15         if(lx!=rx){
16             int m = (lx+rx)/2;
17             build_x(lx, m, ly, ry, idx*2);
18             build_x(m+1, rx, ly, ry, idx*2+1);
19         }
20         build_y(ly, ry, lx, rx, idx);
21     }
22     void build_y(int ly, int ry, int lx, int rx, int idx, int idy=1){
23         iy[idy] = ly;
24         fy[idy] = ry;
25         if(ly == ry){
26             if(lx == rx) value[idx][idy] = v[lx][ly];
27             else value[idx][idy] = fun(value[idx*2][idy], value[idx*2+1][idy]);
28         }
29         else{
30             int m = (ly+ry)/2;
31             build_y(ly, m, lx, rx, idx, idy*2);
32             build_y(m+1, ry, lx, rx, idx, idy*2+1);
33             value[idx][idy] = fun(value[idx][idy*2], value[idx][idy*2+1]);
34         }
35     }
36
37     T queryY(int qly, int qry, int idx=1, int idy=1){
38         if(iy[idy] > qry || fy[idy] < qly) return iden;
39         if(qly <= iy[idy] && fy[idy] <= qry) return value[idx][idy];
40         T response = queryY(qly, qry, idx, idy*2);
41         response = fun(response, queryY(qly, qry, idx, idy*2+1));
42         return response;
43     }
44
45     T queryX(int qlx, int qly, int qrx, int qry, int idx=1, int idy=1){
46         if(ix[idx] > qrx || fx[idx] < qlx) return iden;

```

```

47     if(qlx <= ix[idx] && fx[idx] <= qrx) return queryY(qly, qry, idx, idy);
48     T response = queryX(qlx, qly, qrx, qry, idx*2, idy);
49     response = fun(response, queryX(qlx, qly, qrx, qry, idx*2+1, idy));
50     return response;
51 }
52
53 T updateY(int qiy, T value, int idx=1, int idy=1){
54     if(qiy < iy[idy] || fy[idy] < qiy) return this->value[idx][idy];
55     T response = updateY(qiy, value, idx, idy*2);
56     response = fun(response, updateY(qiy, value, idx, idy*2+1));
57     return this->value[idx][idy] = response;
58 }
59
60 T updateX(int qix, int qiy, T value, int idx=1, int idy=1){
61     if(qix < ix[idx] || fx[idx] < qix) return this->value[idx][idy];
62     if(ix[idx]==fx[idx] && ix[idx]==qix) return updateY(qiy, value, idx, idy);
63     T response = queryX(qix, qiy, value, idx*2, idy);
64     response = fun(response, queryX(qix, qiy, value, idx*2+1, idy));
65     return response;
66 }
67
68 };
69
70 int v[maxn][maxn];
71 Seg2D<int, maxn, maxn, v> seg; // usage

```

Codigo: segtreeLazy.cpp

```

1
2 struct Nodes{
3     ll value;
4     ll lazy;
5     int ini, fim;
6 }nodes[maxn];
7
8 struct Seg{
9     void build(int ini, int fim, int idx=1){
10         nodes[idx].ini = ini;
11         nodes[idx].fim = fim;
12         nodes[idx].lazy = 0;
13         if(ini == fim){
14             nodes[idx].value = 0;
15             return;
16         }
17         int m = (ini + fim)/2;
18         build(ini, m, idx*2);
19         build(m+1, fim, idx*2+1);

```

```

20     nodes[idx].value = nodes[idx*2].value + nodes[idx*2+1].value;
21 }
22 ll query(int ini, int fim, int idx=1){
23     prop(idx);
24     if(nodes[idx].fim < ini || nodes[idx].ini > fim) return 0;
25     if(ini <= nodes[idx].ini && nodes[idx].fim <= fim) return nodes[idx].value;
26     return query(ini, fim, 2*idx) + query(ini, fim, 2*idx+1);
27 }
28 void update(int ini, int fim, int v, int idx=1){
29     prop(idx);
30     if(nodes[idx].fim < ini || nodes[idx].ini > fim) return;
31     if(ini <= nodes[idx].ini && nodes[idx].fim <= fim) {
32         nodes[idx].lazy += v;
33         prop(idx);
34         return;
35     }
36     update(ini, fim, v, 2*idx);
37     update(ini, fim, v, 2*idx+1);
38     nodes[idx].value = nodes[idx*2].value + nodes[idx*2+1].value;
39 }
40 void prop(int idx){
41     if(nodes[idx].lazy == 0) return;
42     if(idx*2 < maxn) nodes[idx*2].lazy += nodes[idx].lazy;
43     if(idx*2+1 < maxn) nodes[idx*2+1].lazy += nodes[idx].lazy;
44     nodes[idx].value += nodes[idx].lazy * (nodes[idx].fim - nodes[idx].ini + 1);
45     nodes[idx].lazy = 0;
46 }
47 }seg;
48
49
50
51
52
53
54
55
56 /*
57 // TODO: Limpar implementação
58 ll values[maxn];
59 ll lazy[maxn];
60
61 template<typename T>
62 struct SegTree{
63     struct Node{
64         int ini, fim;
65     }tree[maxn];
66     T *values;

```

```

67     T *lazy;
68
69     std::vector<T> initValues;
70     T identity;
71
72     T fun(T a, T b){ // operação
73         return a + b;
74     }
75
76     T build(int idx, int ini, int fim);
77     T get(int idx, int ini, int fim);
78     T add(int idx, int ini, int fim, T nvalue);
79
80     T get(int idx, int index){ return get(idx,index,index); }
81     T add(int idx, int index) { add(idx,index,index); }
82
83     void prop(int idx);
84 };
85
86 template<typename T>
87 T SegTree<T>::build(int idx, int ini, int fim){
88     tree[idx].ini = ini;
89     tree[idx].fim = fim;
90     if(ini==fim) return values[idx] = initValues[ini];
91     int m = (ini+fim)/2;
92     return values[idx] = fun(build(idx*2,ini, m), build(idx*2+1, m+1,fim));
93 }
94 template<typename T>
95 T SegTree<T>::get(int idx, int ini, int fim){
96     prop(idx);
97     if(tree[idx].fim < ini || tree[idx].ini > fim) return identity;
98     if(ini <= tree[idx].ini && tree[idx].fim <= fim) return values[idx];
99     return fun(get(2*idx,ini,fim), get(2*idx+1,ini,fim));
100 }
101
102 template<typename T>
103 T SegTree<T>::add(int idx, int ini, int fim, T value){
104     prop(idx);
105     if(tree[idx].fim < ini || tree[idx].ini > fim) return values[idx];
106     if(ini <= tree[idx].ini && tree[idx].fim <= fim) {
107         lazy[idx] = value;
108         return fun(values[idx], value);
109     }
110     return values[idx] = fun(add(2*idx, ini, fim, value), add(2*idx+1,ini,fim,value));
111 }
112 template<typename T>
113 void SegTree<T>::prop(int idx){

```

```

114     if(lazy[idx] == identity) return;
115     values[idx] = fun(values[idx], lazy[idx]);
116     if(2*idx < maxn)
117         lazy[idx*2] = fun(lazy[2*idx], lazy[idx]);
118     if(2*idx+1<maxn)
119         lazy[idx*2+1] = fun(lazy[2*idx+1], lazy[idx]);
120     lazy[idx] = identity;
121 }*/

```

Codigo: smallSeg.cpp

```

1  template<typename T>
2  struct SegTree{
3      static const int len = 2*maxn;
4      T v[len];
5      int n;
6
7      void build(){
8          for(int i=n-1;i>=0;i--){
9              v[i]= min(v[i*2], v[i*2+1]);
10         }
11     }
12
13     int query(int l, int r){
14         r++;
15         l+=n;r+=n;
16         int ans = INF;
17         for(;l<r;l/=2,r/=2){
18             if(l&1) ans = min(ans,v[l++]);
19             if(r&1) ans = min(ans,v[--r]);
20         }
21         return ans;
22     }
23
24     void update(int i, int val){
25         i+=n;
26         v[i].value = val;
27         i/=2;
28         while(i>0){
29             v[i] = min(v[2*i], v[2*i+1]);
30             i/=2;
31         }
32     }
33 };

```

5.5 Sparse Table Disjunta

Sparse Table disjunta

Codigo: disjointSparseTable.cpp

```

1  template<int maxp, int maxn>
2  struct dSparse{
3      int n;
4      int st[maxp][maxn];
5      int nSiz;
6      int nLog;
7      int *v;
8
9      dSparse(){
10     }
11     dSparse(int n, int *v){
12         init(n,v);
13     }
14     void init(int n, int *v){
15         this->v = v;
16         this->n = n;
17     }
18
19     void build(){
20         nSiz = 1;
21         nLog = 0;
22         while(nSiz<n){
23             nLog++;
24             nSiz*=2;
25         }
26         for(int h=0;h<nLog;h++){
27             int blockSiz = nLog - h;
28             for(int block=0;block<<(1<<h);block++){
29                 int ini = block<<blockSiz;
30                 int fin = (block+1)<<blockSiz;
31                 int mid = (ini+fin)/2;
32                 st[h][mid] = v[mid];
33                 st[h][mid-1] = v[mid-1];
34                 for(int i=mid+1;i<fin;i++) st[h][i] = min(st[h][i-1],v[i]);
35                 for(int i=mid-2;i>=ini;i--) st[h][i] = min(st[h][i+1], v[i]);
36             }
37         }
38     }
39 }
40
41 int query(int l, int r){

```

```
42     if(l==r) return st[nLog-1][l];
43     int lev = nLog - 32 + __builtin_clz(l^r);
44     return min(st[lev][l], st[lev][r]);
45 }
46 };
```

Capítulo 6

geometry

6.1 Convex Hull

Convex Hull

Codigo: convexHull.cpp

```
1 bool ccw(ii a, ii b, ii c, bool colinear=false){
2     ii x = {b.x - a.x, b.y - a.y};
3     ii y = {c.x - b.x, c.y - b.y};
4     int val = x.x * y.y - x.y * y.x;
5     return val > 0 || (colinear && val==0);
6 }
7
8 vi convexHul(const vii &points){
9     int n = points.size();
10    vi p(n);
11    for(int i=0;i<n;i++) p[i] = i;
12    sort(all(p), [&](int a, int b){ return points[a] < points[b]; });
13
14    vi h(2*n);
15    int k = 0;
16
17    for(int i=0;i<n;i++){
18        while(k>=2 && !ccw(points[h[k-2]], points[h[k-1]], points[p[i]], true)) k--;
19        h[k++] = p[i];
20    }
```

```
21  for(int i=n-1,t=0;i>=0;--i){
22      while(t>=2 && !ccw(points[h[k-2]], points[h[k-1]], points[p[i]], true)) {
23          k--;
24          t--;
25      }
26      h[k++] = p[i];
27      t++;
28  }
29  h.resize(k);
30  sort(all(h));
31  return h;
32 }
```

Capítulo 7

graph

7.1 Floyd warshall

Floyd Warshall

Codigo: floyd.cpp

```
1  const int maxn = 100;
2  int n;
3  int dist[maxn][maxn];
4  int matr[maxn][maxn];
5  void floyd(){
6      for(int i=0;i<n;i++){
7          for(int j=0;j<n;j++){
8              matr[i][j] = dist[i][j];
9              if(dist[i][j] == -1) matr[i][j] = INF;
10         }
11     }
12
13     for(int k=0;k<n;k++){
14         for(int i=0;i<n;i++){
15             for(int j=0;j<n;j++){
16                 matr[i][j] = min(matr[i][j], matr[i][k]+matr[k][j]);
17             }
18         }
19     }
20 }
```

7.2 Max Flow Min Cost

Max Flow Min Cost

Codigo: maxFlowMinCost.cpp

```

1  const int maxn = 1123;
2  const int maxm = 51234;
3
4  struct Edge{
5      int from, to;
6      ll cap;
7      ll cost;
8  }edges[maxm];
9  int cnt = 0;
10
11 vi prox[maxn];
12
13 ll dist[maxn];
14 ll which[maxn];
15
16 void resetState(int n){
17     for(int i=0;i<=n;i++) prox[i].clear();
18     cnt = 0;
19 }
20
21 void dijkstra(int s, int t){
22     fill(dist, dist+maxn, LINF);
23     priority_queue<pair<ll,int>, vector<pair<ll, int>>, less<pair<ll,int>>> que;
24
25     dist[s] = 0;
26     que.push({0,s});
27
28     while(!que.empty()){
29         auto atu = que.top();
30         que.pop();
31         for(int edNum: prox[atu.y]){
32             int to = edges[edNum].to;
33             if(edges[edNum].cap<=0) continue;
34             ll newDist = atu.x + edges[edNum].cost;
35             if(dist[to] <= newDist) continue;
36             dist[to] = newDist;
37             which[to] = edNum;
38             que.push({dist[to], to});
39         }
40     }
41 }

```

```

42 ll flow(int s, int t, ll K){
43     ll flow = 0;
44     ll cost = 0;
45     while(flow < K){
46         dijkstra(s, t);
47         if(dist[t]>=LINF) break;
48         ll f = K - flow;
49         int curr = t;
50         while(curr != s){
51             int edNum = which[curr];
52             f = min(f, edges[edNum].cap);
53             curr = edges[edNum].from;
54         }
55
56         flow += f;
57         cost += f*dist[t];
58
59         curr = t;
60         while(curr != s){
61             int edNum = which[curr];
62             edges[edNum].cap -= f;
63             edges[edNum^1].cap += f;
64             curr = edges[edNum].from;
65         }
66     }
67     if(flow < K) return -1;
68     return cost;
69 }
70 void _addEdg(int idx, int from, int to, ll cost, ll cap){
71     edges[idx].from = from;
72     edges[idx].to = to;
73     edges[idx].cost = cost;
74     edges[idx].cap = cap;
75 }
76 void add(int a, int b, ll c, ll cap){
77     prox[a].push_back(cnt);
78     _addEdg(cnt, a, b, c, cap);
79     cnt++;
80     prox[b].push_back(cnt);
81     _addEdg(cnt, b, a, -c, 0);
82     cnt++;
83     // Bidirecional, então adiciona mais essas duas
84     prox[b].push_back(cnt);
85     _addEdg(cnt, b, a, c, cap);
86     cnt++;
87     prox[a].push_back(cnt);
88     _addEdg(cnt, a, b, -c, 0);

```

```

89     cnt++;
90 }

```

7.3 Push Relabel

Push Relabel

Codigo: pushRelabelMaxFlow.cpp

```

1  template <int MAXN, class T = int>
2  struct HLPP {
3      const T inf = numeric_limits<T>::max();
4      struct edge { int to, rev; T f; };
5      int s = MAXN - 1, t = MAXN - 2;
6      vector<edge> adj[MAXN];
7      vector<int> lst[MAXN], gap[MAXN];
8      T excess[MAXN];
9      int highest, height[MAXN], cnt[MAXN], work;
10     void addEdge(int from, int to, int f, bool isDirected = true) {
11         adj[from].push_back({to, (int)adj[to].size(), f});
12         adj[to].push_back({from, (int)adj[from].size() - 1, isDirected ? 0 : f});
13     }
14     void updHeight(int v, int nh) {
15         work++;
16         if (height[v] != MAXN) cnt[height[v]]--;
17         height[v] = nh;
18         if (nh == MAXN) return;
19         cnt[nh]++, highest = nh;
20         gap[nh].push_back(v);
21         if (excess[v] > 0) lst[nh].push_back(v);
22     }
23     void globalRelabel() {
24         work = 0;
25         fill(all(height), MAXN);
26         fill(all(cnt), 0);
27         for (int i = 0; i < highest; i++) lst[i].clear(), gap[i].clear();
28         height[t] = 0;
29         queue<int> q({t});
30         while (!q.empty()) {
31             int v = q.front();
32             q.pop();
33             for (auto &e : adj[v])
34                 if (height[e.to] == MAXN && adj[e.to][e.rev].f > 0)
35                     q.push(e.to), updHeight(e.to, height[v] + 1);

```

```

36     highest = height[v];
37 }
38 }
39 void push(int v, edge &e) {
40     if (excess[e.to] == 0) lst[height[e.to]].push_back(e.to);
41     T df = min(excess[v], e.f);
42     e.f -= df, adj[e.to][e.rev].f += df;
43     excess[v] -= df, excess[e.to] += df;
44 }
45 void discharge(int v) {
46     int nh = MAXN;
47     for (auto &e : adj[v]) {
48         if (e.f <= 0) continue;
49         if (height[v] == height[e.to] + 1) {
50             push(v, e);
51             if (excess[v] <= 0) return;
52         } else nh = min(nh, height[e.to] + 1);
53     }
54     if (cnt[height[v]] > 1) {
55         updHeight(v, nh);
56         return;
57     }
58     for (int i = height[v]; i <= highest; i++) {
59         for (auto j : gap[i]) updHeight(j, MAXN);
60         gap[i].clear();
61     }
62 }
63 T calc(int heur_n = MAXN) {
64     fill(all(excess), 0);
65     excess[s] = inf, excess[t] = -inf;
66     globalRelabel();
67     for (auto &e : adj[s]) push(s, e);
68     for (; highest >= 0; highest--) {
69         while (!lst[highest].empty()) {
70             int v = lst[highest].back();
71             lst[highest].pop_back();
72             discharge(v);
73             if (work > 4 * heur_n) globalRelabel();
74         }
75     }
76     return excess[t] + inf;
77 }
78 };

```

Capítulo 8

math

8.1 Brent

Brent

Codigo: brent.cpp

```
1 // Precisa dos métodos:
2 // utils::{mul, rand, binPow}
3 // millerRabin::isPrime
4
5 ll f(ll x, ll c, ll mod){
6     return (mul(x,x,mod) + c)%mod;
7 }
8
9 ll brent(ll n){
10     if(isPrime(n)) return n;
11     for(int w: {2,3,5,7,11,13,17,19})
12         if(n%w==0) return w;
13     ll x0 = (rnd()%(n-1)) + 1;
14     ll c = (rnd()%(n-1))+1;
15     ll turt = x0;
16     ll rabbit = f(turt, c, n);
17     ll g = 1;
18
19     ll lastRabbit = rabbit;
20     int l = 1;
```

```

21  int m = 128;
22  while(g==1){
23      turt = rabbit;
24      for(int i=0;i<l;i++){
25          rabbit = f(rabbit, c, n);
26
27          ll q = 1;
28          for(int k=0;k<l;k+=m){
29              lastRabit = rabbit;
30              int lim = min(m, l-k);
31              for(int i=0;i<lim; i++){
32                  rabbit = f(rabbit, c, n);
33                  q = mul(q, abs(rabbit - turt), n);
34              }
35              g = __gcd(q, n);
36              if(g!=1) break;
37          }
38          l *= 2;
39      }
40      if(g == n){
41          g = 1;
42          while(g == 1){
43              lastRabit = f(lastRabit, c, n);
44              g = __gcd(abs(lastRabit - turt), n);
45          }
46      }
47      return g;
48  }

```

8.2 FFT

FFT

Codigo: fastFft.cpp

```

1  typedef complex<double> Cd;
2  typedef complex<long double> Cld;
3
4  void fft(vector<Cd> &a) {
5      int n = a.size(), L = 31 - __builtin_clz(n);
6      static vector<Cld> R(2, 1);
7      static vector<Cd> rt(2, 1); // (~ 10% faster if double)
8      for (static int k = 2; k < n; k *= 2) {
9          R.resize(n);

```

```

10     rt.resize(n);
11     auto x = polar(1.0L, acos(-1.0L) / k);
12     for(int i=k; i<2 * k;i++) rt[i] = R[i] = i & 1 ? R[i / 2] * x : R[i / 2];
13 }
14 vi rev(n);
15 for(int i=0;i<n;i++) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
16 for(int i=0;i<n;i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
17 for (int k = 1; k < n; k *= 2)
18     for (int i = 0; i < n; i += 2 * k) for(int j=0;j<k;j++) {
19         // C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled) ///
20         // include-line
21         auto x = (double *)&rt[j + k],
22              y = (double *)&a[i + j + k]; /// exclude-line
23         Cd z(x[0] * y[0] - x[1] * y[1],
24             x[0] * y[1] + x[1] * y[0]); /// exclude-line
25         a[i + j + k] = a[i + j] - z;
26         a[i + j] += z;
27     }
28 }
29
30 template <int M> vi convMod(const vi &a, const vi &b) {
31     if (a.empty() || b.empty()) return {};
32     vi res((a.size()) + (b.size()) - 1);
33     int B = 32 - __builtin_clz((res.size())), n = 1 << B, cut = int(sqrt(M));
34     vector<Cd> L(n), R(n), outs(n), outl(n);
35     for(int i=0;i<(a.size());i++) L[i] = Cd((int)a[i] / cut, (int)a[i] % cut);
36     for(int i=0;i<(b.size());i++) R[i] = Cd((int)b[i] / cut, (int)b[i] % cut);
37     fft(L), fft(R);
38     for(int i=0;i<n;i++) {
39         int j = -i & (n - 1);
40         outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
41         outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
42     }
43     fft(outl), fft(outs);
44     for(int i=0;i<(res.size());i++) {
45         ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5);
46         ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
47         res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
48     }
49     return res;
50 }

```

Codigo: fft.cpp

```

1 typedef complex<double> Cd;
2
3 // FFT

```

```

4 void get_roots(bool f, int n, vector<Cd> &roots){
5     const static double PI = acosl(-1);
6     for(int i=0;i<n/2;i++){
7         double alpha = i*((2*PI)/2);
8         if(f) alpha = -alpha;
9         roots[i] = {cos(alpha), sin(alpha)};
10    }
11 }
12
13 //NTT
14 template<int p>
15 void get_roots(bool f, int n, vector<mod_int<p>> &roots){
16     mod_int<p> r;
17     int ord;
18     if(p==998244353){
19         r = 102292;
20         ord = (1<<23);
21     } else if(p==754974721){
22         r = 739831874;
23         ord = (1<<24);
24     } else if(p==167772161){
25         r = 243;
26         ord = (1<<25);
27     } else assert(false);
28
29     if(f) r = r^(p - 1 - ord/n);
30     else r = r^(ord/n);
31     roots[0] = 1;
32     for(int i=1;i<n/2;i++){
33         roots[i] = roots[i-1]*r;
34     }
35
36 template<typename T>
37 void fft(vector<T> &a, bool f, int N, vector<int> &rev){
38     for(int i=0;i<N;i++) if(i<rev[i])
39         swap(a[i], a[rev[i]]);
40     int l, r, m;
41     vector<T> roots(N);
42     for(int n=2;n<=N;n*=2){
43         get_roots(f, n, roots);
44         for(int pos=0;pos<N;pos+=n){
45             l = pos + 0, r=pos+n/2, m=0;
46             while(m<n/2){
47                 auto t = roots[m]*a[r];
48                 a[r] = a[l] - t;
49                 a[l] = a[l] + t;
50                 l++, r++, m++;

```

```

51     }
52 }
53 }
54 if(f){
55     auto invN = T(1) / T(N);
56     for(int i=0;i<N;i++){
57         a[i] = a[i] * invN;
58     }
59 }
60
61 template<typename T>
62 vector<T> convolution(vector<T> &a, vector<T> &b){
63     vector<T> l(a.begin(), a.end()), r(b.begin(), b.end());
64     int N = l.size() + r.size() - 1;
65     int n = 1, log_n = 0;
66     while(n <= N) n*=2, log_n++;
67     vector<int> rev(n);
68     for(int i=0;i < n;i++){
69         rev[i] = 0;
70         for(int j=0;j<log_n;j++) if(i>>j&1)
71             rev[i] |= 1<<(log_n-1-j);
72     }
73     assert(N <= n);
74     l.resize(n);
75     r.resize(n);
76     fft(l, false, n, rev);
77     fft(r, false, n, rev);
78     for(int i=0;i<n;i++){
79         l[i] *= r[i];
80     }
81     fft(l, true, n, rev);
82     l.resize(N);
83     return l;
84 }
85
86 template<int p, typename T>
87 vector<mod_int<p>> ntt(vector<T> &a, vector<T> &b){
88     vector<mod_int<p>> A(a.begin(), a.end()), B(b.begin(), b.end());
89     return convolution(A, B);
90 }
91
92 // Convolucao de inteiro
93 //
94 // Precisa do CRT
95 //
96 // Tabela de valores:
97 // [0, 1] - <int, 1>

```

```

98 // [-1e5, 1e5] - <11, 2>
99 // [-1e9, 1e9] - <__int128, 3>
100 template <typename T, int mods>
101 vector<T> int_convolution(vector<int> &a, vector<int> &b){
102     static const int M1 = 998244353 , M2 = 754974721 , M3 = 167772161;
103
104     auto c1 = ntt<M1>(a, b);
105     auto c2 = (mods >= 2? ntt<M2>(a, b): vector<mod_int<M2>>());
106     auto c3 = (mods >= 3? ntt<M3>(a, b): vector<mod_int<M3>>());
107
108     vector<T> ans;
109     for(int i=0;i < c1.size();i++){
110         crt<T> at(c1[i].v, M1);
111         if(mods >= 2) at = at*crt<T>(c2[i].v, M2);
112         if(mods >= 3) at = at*crt<T>(c3[i].v, M3);
113         ans.push_back(at.a);
114         if(at.a > at.m/2) ans.back() -= at.m;
115     }
116     return ans;
117 }

```

8.3 Fermat

Pequeno Teorema de Fermat

Codigo: fermatPrimeTest.cpp

```

1 // Precisa dos métodos:
2 // utils::{mul, rand, binPow}
3
4 bool isPrime(ll p){
5     if(p < 4) return p==2 || p == 3;
6     int it = 10;
7     while(it--){
8         ll a = ((rnd())%(p-3)) + 2;
9         if(binPow(a,p-1, p) != 1) return false;
10    }
11    return true;
12 }

```

8.4 Miller Rabin

Miller Rabin

Codigo: millerRabin.cpp

```

1 // Precisa dos métodos:
2 // utils::{mul, binPow}
3
4 bool check(ll a, ll d, ll n, int s){
5     ll x = binPow(a, d, n);
6     if(x == 1 || x == n-1) return true;
7     for(int r=0;r<s;r++){
8         x = mul(x, x, n);
9         if(x == n - 1) return true;
10    }
11    return false;
12 }
13 bool isPrime(ll n){
14     if(n<=3) return n==2 || n==3;
15     int s = 0;
16     ll d = n-1;
17     while(~d&1) {
18         d = d>>1;
19         s++;
20     }
21     for(ll w: {2, 3, 5, 13, 19, 73, 193, 407521, 299210837}) if(n == w) return true;
22     for(ll w: {2, 325, 9375, 28178, 450775, 9780504, 1795265022}){
23         if(!check(w, d, n, s)) return false;
24     }
25     return true;
26 }

```

8.5 Pollard Rho

Pollard Rho

Codigo: pollardRho.cpp

```

1 // Precisa dos métodos:
2 // millerRabin::isPrime
3 // utils::{mul, rand}
4

```

```

5 ll f(ll x, ll c, ll mod){
6     return (mul(x,x,mod) + c)%mod;
7 }
8
9 ll rho(ll n) {
10     if(isPrime(n)) return n;
11     for(int w: {2,3,5,7,11,13,17,19})
12         if(n%w==0) return w;
13     ll x0 = (rnd()%(n-2)) + 2;
14     ll c = (rnd()%(n-1)) + 1;
15     ll x = x0;
16     ll y = x0;
17     ll g = 1;
18     while (g == 1) {
19         x = f(x, c, n);
20         y = f(y, c, n);
21         y = f(y, c, n);
22         g = __gcd(abs(x - y), n);
23         if(g == n) return rho(n);
24     }
25     return g;
26 }

```

8.6 Teorema chines

Teorema do resto chines

Codigo: crt.cpp

```

1 template<typename T>
2 tuple<T, T, T> ext_gcd(T a, T b){
3     if(!a) return {b, 0, 1};
4     auto [g, x, y] = ext_gcd(b%a, a);
5     return {g, y-b/a*x, x};
6 }
7 template<typename T=ll>
8 struct crt{
9     T a, m;
10
11     crt() : a(0), m(1) {}
12     crt(T a, T m) : a(a), m(m) {}
13
14     crt operator*(crt C) {
15         auto [g, x, y] = ext_gcd(m, C.m);

```

```

16         if((a - C.a) % g) a=-1;
17         if(a == -1 or C.a == -1) return crt(-1, 0);
18         T lcm = m/g*C.m;
19         T ans = a + (x*(C.a - a)/g % (C.m/g)) * m;
20         return crt((ans % lcm + lcm) % lcm, lcm);
21     }
22 };

```

8.7 Utils

Utils

Codigo: utils.cpp

```

1 ll rnd(){
2     static mt19937_64 rn(chrono::system_clock::now().time_since_epoch().count());
3     ll r = rn();
4     if(r<0) r = -r;
5     return r % (1LL<<62);
6 }
7
8 // Preferir essa implementacao
9 ll mul(ll a, ll b, ll m){
10     return (__int128(a) * b)%m;
11 }
12 /*
13 ll mul(ll a, ll b, ll m){
14     ll result = 0;
15     while(b){
16         if(b&1) result = (result + a)%m;
17         a = (a+a)%m;
18         b>>=1;
19     }
20     return result;
21 */
22
23 ll binPow(ll base, ll ex, ll mod){
24     if(ex==0) return 1;
25     if(ex==1) return base;
26     ll ans = binPow(base, ex/2, mod);
27     ans = mul(ans, ans, mod);
28     if(ex%2) ans = mul(ans, base, mod);
29     return ans;
30 }

```

Capítulo 9

strings

9.1 Suffix Array

Suffix Array

Codigo: suffixArray.cpp

```
1 vi makeSa(const string &s){
2     int n = s.size();
3     int N = max(n,300);
4     vi sa(n), ra(n);
5     for(int i=0;i<n;i++){
6         sa[i] = i;
7         ra[i] = s[i];
8     }
9     vi cnt(N,0);
10    vi nsa(sa), nra(n);
11    for(int k=0;k<n;k=(k?k*2:k+1)){
12        fill(all(cnt), 0);
13        for(int i=0;i<n;i++){
14            nsa[i] = (sa[i] - k + n)%n;
15            cnt[ra[i]]++;
16        }
17        for(int i=1;i<N;i++) cnt[i] += cnt[i-1];
18        for(int i=n-1;i>=0;i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];
19
20    int r = 0;
```

```

21     nra[sa[0]] = r;
22     for(int i=1;i<n;i++){
23         nra[sa[i]] = r += ra[sa[i]] != ra[sa[i-1]] or ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
24     }
25     swap(nra, ra);
26     if(ra[sa[n-1]]==n-1) break;
27 }
28 return sa;
29 }
30
31 vi kasai(const string &s, const vi &sa){
32     int n = s.size();
33     vi lcp(n,0), ra(n);
34     for(int i = 0;i<n;i++) ra[sa[i]] = i;
35     int k = 0;
36     for(int i=0;i<n;i++, k -= !!k){
37         if(ra[i]==n-1){
38             k = 0;
39             continue;
40         }
41         int j = sa[ra[i]+1];
42         while(i+k < n and j+k < n and s[i+k]==s[j+k]) k++;
43         lcp[ra[i]] = k;
44     }
45     return lcp;
46 }

```